

Module Two Milestone: Code Review (Transcript)

Justin M Davis

Southern New Hampshire University

CS-499 Computer Science Capstone

Joseph Martinez

07/19/2026

Module Two Milestone: Code Review (Transcript)

CS 499 Milestone One Code Review Video Script

Estimated length: 10–14 minutes

Before Recording

Open these windows:

1. Browser showing Travlr Getaways customer site.
2. Browser showing Angular admin app.
3. Code editor opened to the project root.
4. Terminal optional, showing folder hierarchy if useful.

Open these files in tabs before starting:

- app.js
- app_api/routes/index.js
- app_api/controllers/trips.js
- app_api/models/travlr.js
- app_api/models/user.js
- app_admin/src/app/services/trip-data.ts
- app_admin/src/app/app.routes.ts
- app_admin/src/app/edit-trip/edit-trip.ts

Recording Script

Opening

[SHOW: Travlr Getaways customer-facing site in browser.]

Say:

Hello, my name is Justin Davis. For my CS 499 capstone artifact, I selected the Travlr Getaways full-stack web application that I created in CS 465: Full Stack Development. This application includes a customer-facing website, an Angular administrative interface, an Express and Node.js API, MongoDB persistence through Mongoose, and JWT-based authentication for administrative features.

In this informal code review, I will review the existing code and explain planned enhancements in three categories: software engineering and design, algorithms and data structures, and databases. I will also discuss security, maintainability, defensive programming, and areas where the current implementation can be improved.

Project Overview

[SHOW: Customer-facing travel page.]

Say:

The customer-facing portion of the application displays available travel packages. This part of the application uses server-rendered pages through Express and Handlebars.

[SWITCH TO: Angular admin app.]

Say:

The administrative portion is an Angular single-page application. It allows an authenticated user to view trips and perform administrative actions such as adding or editing trip records.

[SWITCH TO: Code editor. Show folder hierarchy.]

Point cursor to:

- app_server
- app_api
- app_admin
- public

Say:

The project is organized into three major areas. The app_server folder contains the server-rendered customer-facing portion of the site. The app_api folder contains the Express API routes, controllers, models, database configuration, and authentication configuration. The app_admin folder contains the Angular administrative application. The public folder contains earlier static assets from the original version of the site.

This organization already demonstrates a useful separation between the customer site, the API, and the Angular administrative front end. However, as I review the code, I will point out places where the internal structure can be improved further.

Category One: Software Engineering and Design

[OPEN: app.js.]

Point cursor to route imports near top:

- indexRouter
- usersRouter

- travelRouter
- apiRouter

Say:

For software engineering and design, I will start with app.js. This file configures the Express application, imports the customer-facing routes, imports the API routes, initializes Passport authentication, connects the database, configures Handlebars, and wires middleware into the request pipeline.

[Point to app.use lines.]

Point cursor to:

```
app.use('/', indexRouter);
```

```
app.use('/users', usersRouter);
```

```
app.use('/travel', travelRouter);
```

```
app.use('/api', apiRouter);
```

Say:

These lines show the main route structure. Requests to the root, users, and travel paths go through the server-rendered portion of the site, while requests beginning with /api are handled by the API router. This is a functional design because it separates the customer pages from the JSON API used by the Angular application.

[Point to CORS block.]

Say:

This block enables CORS for the Angular administrative application running on localhost port 4200. For the original coursework project, this is reasonable because the Angular app and Express API were run separately during development. In a production environment, this would need to be configurable rather than hard-coded, but for this artifact it clearly shows how the front end communicates with the back end.

[OPEN: app_api/routes/index.js.]

Say:

Next, I am looking at the API route file. This file maps API endpoints to controller functions and also defines the JWT authentication middleware used to protect administrative actions.

[Point to register/login routes.]

Point cursor to:

```
router.route("/register").post(authController.register);
```

```
router.route("/login").post(authController.login);
```

Say:

The register and login endpoints are public because users need to be able to create an account and authenticate before receiving a token.

[Point to trips routes.]

Point cursor to:

```
.route("/trips")
```

```
.get(tripsController.tripsList)
```

```
.post(authenticateJWT, tripsController.tripsAddTrip);
```

Say:

The trips endpoint supports GET for listing trips and POST for adding a new trip. The POST operation is protected with authenticateJWT, which means only authenticated users should be able to add records.

[Point to tripCode route.]

Say:

The individual trip route supports GET by trip code and PUT for updating an existing trip. The update operation is also protected by the JWT middleware.

Existing Functionality Summary

Say:

Overall, the existing design provides a working route-to-controller flow. A request comes into an API route, the route calls a controller, and the controller interacts with the Mongoose model to read or write MongoDB data. That functionality meets the original project requirements, but there are some design and security concerns that can be improved.

Software Engineering Finding One: Controller Responsibilities

[OPEN: app_api/controllers/trips.js.]

Point cursor to imports and Model declaration.

Say:

In trips.js, the controller imports the Mongoose model and directly performs database operations. For example, tripsList calls Model.find, tripsFindByCode calls Model.find, tripsAddTrip creates a new Trip object, and tripsUpdateTrip calls findOneAndUpdate.

[Point to tripsList function.]

Say:

This works, but the controller is responsible for several different concerns at once. It handles the HTTP request and response, decides which database query to run, performs the database operation, and determines the response status. For a small project this is manageable, but as the application grows, this structure can become harder to maintain and test.

Code Review Checklist Connection

Say:

From the code review checklist, this relates to structure, module complexity, repeated code, defensive programming, and maintainability. The code does implement the original design, but the modules could be better separated so that each layer has a clearer responsibility.

Planned Software Engineering Enhancement

Say:

My planned software engineering enhancement is to refactor the API into a clearer layered architecture.

[Optional: show simple text diagram in notes or say verbally.]

Say:

The enhanced flow will be:

Route to controller, controller to service, service to repository or data access layer, repository to Mongoose model, and then MongoDB.

The route layer will define endpoints. The controller layer will handle HTTP request and response concerns. The service layer will handle business rules, validation, and authorization decisions. The repository or data access layer will handle database queries. This will make the code easier to maintain, easier to test, and easier to expand during the later enhancements.

Course Outcomes Connection

Say:

This enhancement supports the course outcomes related to professional communication, computing practices that deliver value, secure software design, and collaborative environments. A clearer architecture makes the code easier for another developer or stakeholder to understand, and it also helps centralize security and validation logic.

Software Engineering Finding Two: JWT Middleware Logic

[RETURN TO: app_api/routes/index.js.]

Point cursor to authenticateJWT function.

Say:

A more specific code review finding is in the authenticateJWT middleware. This function checks for an Authorization header, extracts the bearer token, and verifies it using the JWT secret.

[Point to jwt.verify block and next().]

Say:

The main issue is that next is called after jwt.verify, outside the verification callback. That means the request can continue through the middleware even though the verification result is handled asynchronously in the callback. The intended behavior is that next should only be called after the token has been successfully verified.

[Point to return res.sendStatus(401).json line.]

Say:

There is also a smaller response-handling issue here. sendStatus sends the response immediately, so chaining json after sendStatus is not the correct pattern. This is the kind of logic and defensive programming issue that code review is intended to catch.

Say:

In the enhanced version, I plan to move authentication into a clearer middleware module and ensure that protected routes continue only after successful token verification. This supports the security course outcome because it anticipates unauthorized access and strengthens the application's authentication boundary.

Software Engineering Finding Three: Angular Route Protection

[OPEN: app_admin/src/app/app.routes.ts.]

Point cursor to routes.

Say:

On the Angular side, the route configuration allows direct navigation to add-trip and edit-trip. The server API protects the POST and PUT requests, but the front-end routes themselves are not guarded.

Say:

This is not a complete security failure because the API still requires a valid token for protected write operations. However, from a user experience and defense-in-depth perspective, the client application should also prevent unauthenticated users from reaching administrative forms. As part of the software engineering and security improvements, I can add route guards so the front end better reflects the server-side authorization model.

Category Two: Algorithms and Data Structures

[OPEN: app_api/controllers/trips.js.]

Point cursor to tripsList.

Say:

For the algorithms and data structures category, I am focusing on how the application currently retrieves and presents trip data. The existing tripsList function uses Model.find with an empty filter. This returns all trips from the database.

[Point to .find({}).]

Say:

For the original assignment, this was sufficient because the application only needed to display the available trips. However, this approach does not provide search, filtering, sorting, or ranking. As the trip catalog grows, returning and displaying all trips becomes less useful to the user.

[OPEN: app_admin/src/app/services/trip-data.ts.]

Point cursor to getTrips.

Say:

On the Angular side, the TripData service has a getTrips method that calls the /api/trips endpoint and returns an observable array of trips.

[OPEN: app_admin/src/app/trip-listing/trip-listing.ts, if open; otherwise mention.]

Point cursor to getStuff or describe if not open.

Say:

The trip listing component subscribes to that result and displays the records. The current data flow is straightforward: the Angular component asks the service for all trips, the service calls the API, the API retrieves all records, and the component displays them.

Existing Functionality Summary

Say:

The existing code provides basic list retrieval, which is an important foundation. However, the algorithmic behavior is limited because the application does not help users narrow or prioritize results.

Code Review Checklist Connection

Say:

From the code review checklist, this relates to efficiency, functionality, logic, and whether storage or data use is efficient. There is nothing wrong with retrieving all records for a small dataset, but a more complete travel application should support more useful data access patterns.

Planned Algorithms and Data Structures Enhancement

Say:

My planned enhancement is to add improved trip discovery. The enhanced application will support search, filtering, sorting, and ranking of trips. For example, users could filter by resort, destination, price range, duration, or keyword. The application could then rank results based on relevance.

Say:

One design decision will be whether to perform filtering in MongoDB, in the API service layer, or in the Angular front end. Filtering in the database is better for reducing unnecessary data transfer. In-memory ranking can be useful after the candidate set has already been narrowed. This creates a practical trade-off between simplicity, performance, and flexibility.

Say:

For example, instead of always calling find with an empty filter, the enhanced API could build a query based on optional request parameters. After retrieving candidate trips, the service layer could apply a scoring function that ranks exact matches higher than partial matches. If the catalog grows larger, indexed fields such as code, name, resort, or start date can help make lookup and filtering more efficient.

Suggested Pseudocode to Explain Verbally

Say:

The basic algorithm will follow this structure:

Receive search criteria.

Build a query from required filters.

Retrieve matching trips.

Score each trip based on keyword or preference matches.

Sort results by score or selected sort option.

Return the ordered result set.

This demonstrates algorithmic thinking because the enhancement requires selecting data structures and processing steps based on how the application will actually use trip data.

Course Outcomes Connection

Say:

This enhancement supports the course outcome involving algorithmic principles and design trade-offs. It also supports the outcome involving computing tools that deliver value because the enhanced search experience makes the application more useful to customers and administrators.

Category Three: Databases

[OPEN: app_api/models/travlr.js.]

Point cursor to tripSchema.

Say:

For the database category, I am reviewing the current MongoDB and Mongoose data model. The existing application defines a trip schema with fields such as code, name, length, start date, resort, price per person, image, and description.

[Point to code and name indexes.]

Say:

The code and name fields are indexed. This is useful because trip code is used as an identifier in the application, and name may be useful for lookup or display.

Existing Functionality Summary

Say:

The current database layer supports storing and retrieving trip records. This satisfies the original application requirements, but the data model is still fairly basic. It represents trips, but it does not yet represent customer bookings, reservations, ownership of user data, or administrative reporting.

Database Finding One: Trip Code Uniqueness and Lookup

[OPEN: `app_api/controllers/trips.js`.]

Point cursor to `tripsFindByCode`.

Say:

In the controller, `tripsFindByCode` uses `find` with the trip code. Since the application treats trip code as a unique identifier, `findOne` would better express the intent. On the Angular side, this currently results in the edit form using `value[0]`, which shows that the API is returning an array even though the application expects one record.

[OPEN: `app_admin/src/app/edit-trip/edit-trip.ts`.]

Point cursor to `patchValue(value[0])`.

Say:

Here in the edit trip component, the form is patched using `value[0]`. This works, but it is a sign that the API response shape and the application's actual expectations are not perfectly aligned. Returning a single trip object would make the API cleaner and the front-end code simpler.

Database Finding Two: Data Types

[RETURN TO: `app_api/models/travlr.js`.]

Point cursor to perPerson and length.

Say:

Another database design issue is that perPerson and length are stored as strings. This is functional for display, but it limits future filtering, sorting, and calculations. For example, if I want to filter trips by price range or sort trips by cost, storing the price as a numeric type would be more appropriate. This connects directly to the algorithms enhancement because better data types support better search and sorting behavior.

Database Finding Three: User Model and Authentication

[OPEN: app_api/models/user.js.]

Point cursor to hash/salt and setPassword.

Say:

The user model stores an email, name, password hash, and salt. The password is not stored directly. Instead, the setPassword method generates a salt and stores a PBKDF2 hash. The validPassword method hashes the entered password using the stored salt and compares it to the saved hash.

Say:

This is a good security practice because plaintext passwords are not stored. The user model also generates JWT tokens with an expiration time. That provides a foundation for secure authentication, although the middleware logic reviewed earlier should be improved.

Planned Database Enhancement

Say:

My planned database enhancement is to expand the application from basic trip management into a more complete travel system by adding a booking or reservation model.

The new booking model may include fields such as booking code, user ID, trip code, number of travelers, booking date, status, and total price.

Say:

This enhancement will demonstrate MongoDB document modeling, Mongoose schema validation, indexes, and relationships between users, trips, and bookings. It will also allow administrative reporting through aggregation, such as grouping bookings by trip to show reservation counts or estimated revenue.

Security and Authorization in Database Enhancement

Say:

This database enhancement will also support the security outcome because booking data belongs to specific users. The API should not only check whether a user has a valid token. It should also check whether the authenticated user owns the booking record they are requesting or modifying. Administrators may have broader access, but normal users should only be able to access their own reservations.

Say:

This addresses a common web application vulnerability where a user changes an ID in a request to access another user's data. By adding ownership checks at the service or controller level, the enhanced application will better protect private user data.

Course Outcomes Connection

Say:

This enhancement supports the database category, the security outcome, the professional communication outcome, and the outcome involving computing solutions that deliver industry-specific value. A travel application becomes more realistic and valuable when it can manage reservations rather than only display trip information.

Testing and Documentation Considerations

[SHOW: project structure or any spec files briefly.]

Say:

Another area I will consider during the enhancements is testing and documentation. The Angular project includes spec files, and the existing code contains comments explaining some behavior. However, as the architecture becomes more layered, tests and documentation should be updated to reflect the new design. Code review is not only about finding broken code. It is also about making sure the code remains understandable and maintainable for future updates.

Overall Enhancement Summary

[SHOW: project folder hierarchy or home page.]

Say:

To summarize, the TravlR Getaways application is a strong capstone artifact because it already integrates several areas of computer science: front-end development, back-end development, database persistence, authentication, and API communication.

For software engineering and design, I plan to refactor the API into a clearer layered architecture with better separation between routes, controllers, services, and data access logic.

For algorithms and data structures, I plan to add improved trip discovery through search, filtering, sorting, and ranking, while evaluating the trade-offs between database queries and in-memory processing.

For databases, I plan to extend the MongoDB model by adding customer bookings or reservations, including validation, indexes, aggregation, and resource-level authorization.

Closing

Say:

This code review helped identify practical limitations in the existing application and connect those limitations to the enhancements I plan to complete during the capstone. The original project meets its coursework goals, but the planned enhancements will make it more maintainable, more secure, more useful, and more representative of professional full-stack software development.

End recording.